Registration Login App

1. Title Page

- **App Name**: Registration Login App
- **Student ID**: 20223802028
- **Name**: Chengxuan Li
- **Completion Date**: November 3, 2024

---

2. Basic Design for the APP

i. Written Description of Key Code

- **MainActivity.kt** (Login Interface):
  - `MainActivity` manages the login and registration actions. When the user clicks on the "LOGIN" button, the app checks if the user has already registered. If not, it shows a toast message prompting the user to register first.
  - `MainActivity` uses `SharedPreferences` to store and retrieve user registration details (username and password). If the user's input matches the stored credentials, the app navigates to `LoginSuccessActivity` to display a welcome message.
  - The code includes logic to track login attempts. If the user enters incorrect credentials three times, the login button is disabled to prevent further attempts.
  - key code

```
loginButton.setOnClickListener {
    if (loginAttempts >= 3) {
        loginButton.isEnabled = false
        Toast.makeText(this, "Too many attempts. Please try later.",
Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }

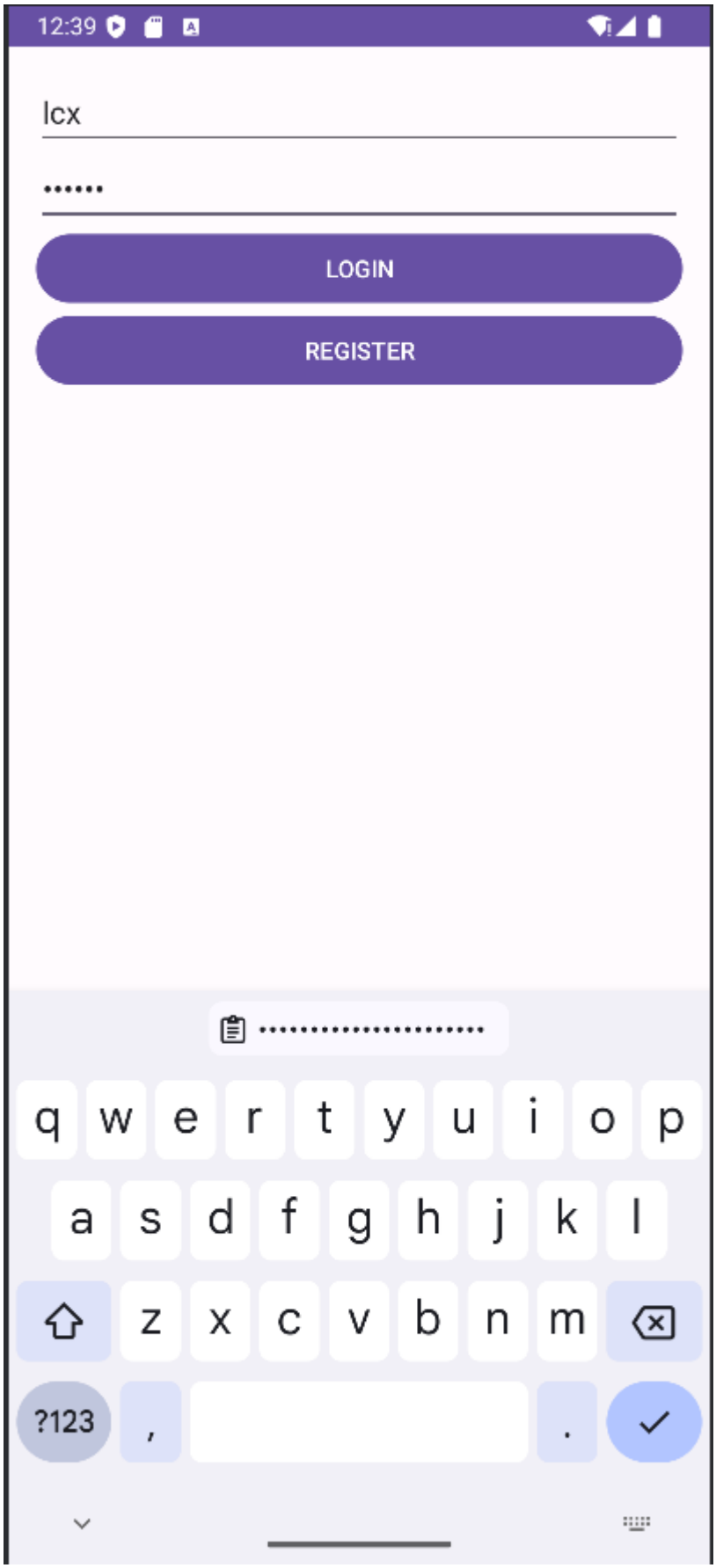
    val username = usernameEditText.text.toString()
    val password = passwordEditText.text.toString()

    val storedUsername = sharedPreferences.getString("username", null)
    val storedPassword = sharedPreferences.getString("password", null)

    if (username == storedUsername && password == storedPassword) {
        val intent = Intent(this, LoginSuccessActivity::class.java)
        intent.putExtra("username", username)
```

```
            startActivity(intent)
        } else {
            loginAttempts++
            Toast.makeText(this, "Incorrect username or password.",
    Toast.LENGTH_SHORT).show()
            if (loginAttempts == 3) {
                loginButton.isEnabled = false
            }
        }
    }
```

screenshoot



- **RegisterActivity.kt** (Registration Interface):
  - `RegisterActivity` allows users to input a username and password, and click "REGISTER" to complete registration.
  - The username and password are saved in `SharedPreferences` for future verification during login.

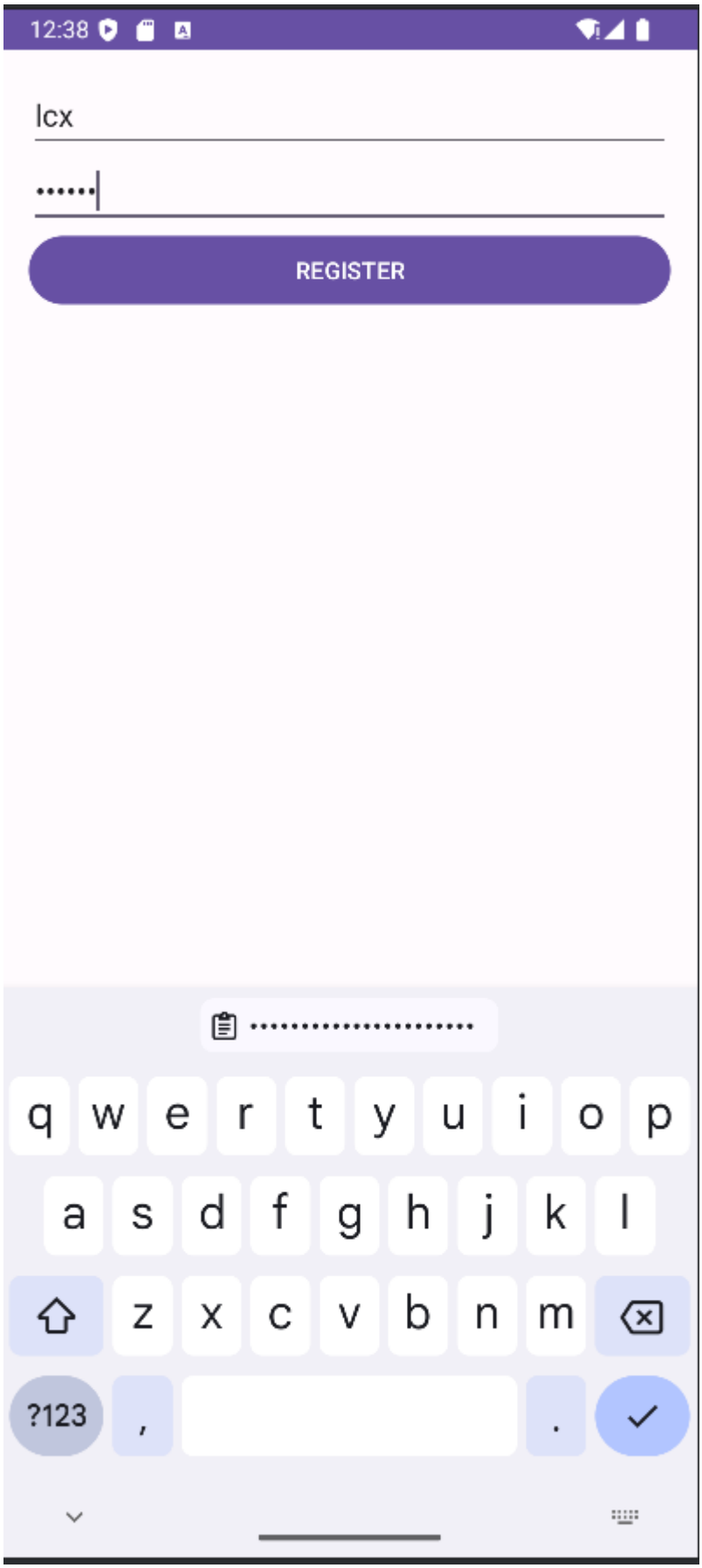
- Upon successful registration, the app redirects the user back to `MainActivity` for login.
- key code

```
registerButton.setOnClickListener {
    val username = usernameEditText.text.toString()
    val password = passwordEditText.text.toString()

    val editor = sharedPreferences.edit()
    editor.putString("username", username)
    editor.putString("password", password)
    editor.apply()

    val intent = Intent(this, MainActivity::class.java)
    startActivity(intent)
    finish()
}
```

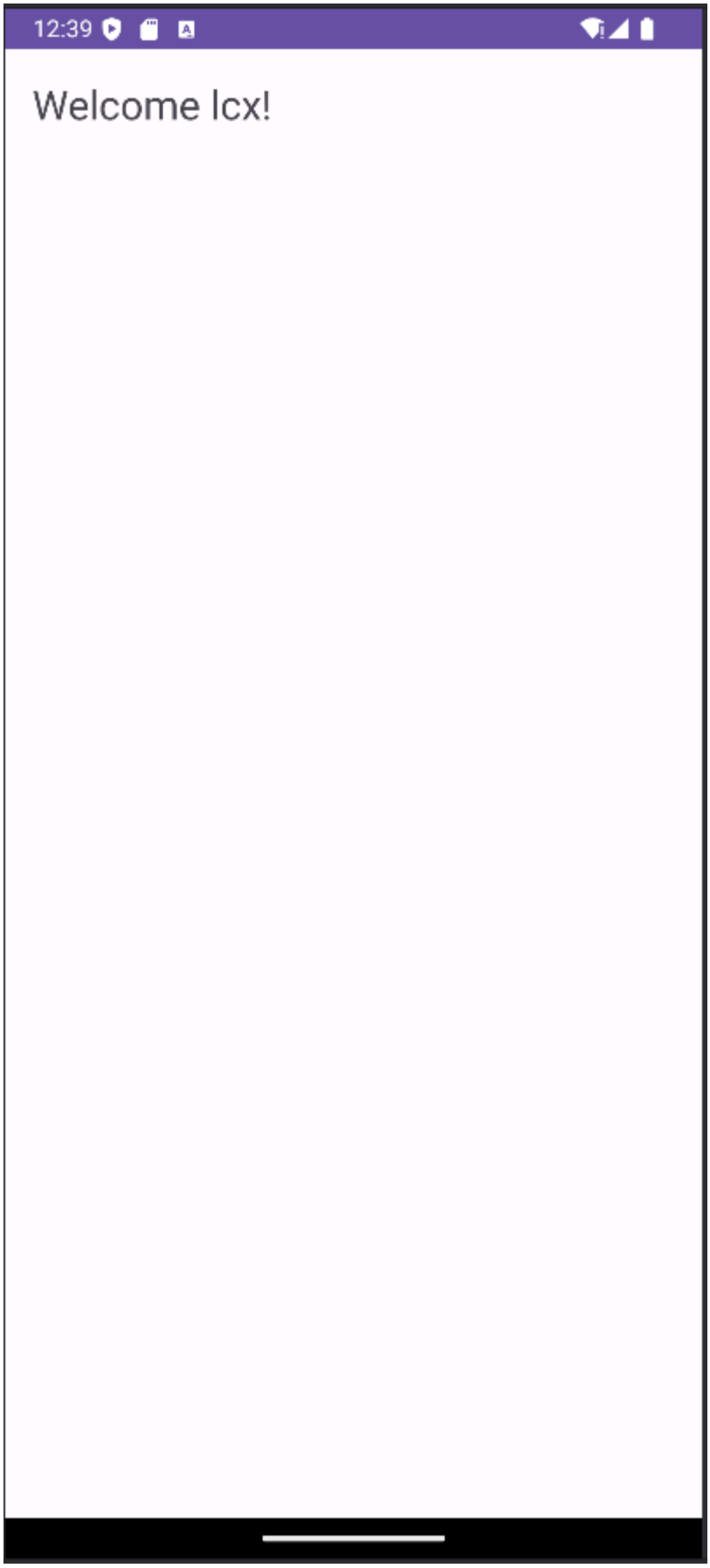
screenshoot



- **LoginSuccessActivity.kt** (Login Success Interface):
  - `LoginSuccessActivity` is triggered when the user successfully logs in. It receives the username from `MainActivity` and displays a welcome message, "Welcome + username."
  - **key code**

```
val username = intent.getStringExtra("username")
welcomeTextView.text = "Welcome $username!"
```

screenshoot



ii. Comment on the Implementation of Tasks

- Use of SharedPreferences: `SharedPreferences` is used to store the user's username and password. This method is suitable for small, simple data storage needs but is less secure compared to a database, making it more appropriate for testing or development purposes.
- Login Attempt Counter: To prevent brute-force login attempts, a counter (`loginAttempts`) tracks the number of incorrect login attempts. If the user fails three times, the login button is disabled, and a message is displayed.
- Activity Navigation: The app uses `Intent` to navigate between `MainActivity` and `RegisterActivity` , as well as to `LoginSuccessActivity` upon successful login. It uses `putExtra` to pass the username to `LoginSuccessActivity` for a personalized welcome message.

---

3. Problems and Solutions Encountered in Implementation

Problems and Solutions

- Problem 1: How to share user data across different activities.
  - Solution: `SharedPreferences` was used to store the user's registration data, allowing all activities to access and verify this information.
- Problem 2: How to disable the login button after three failed attempts.
  - Solution: A `loginAttempts` variable was implemented to count failed attempts. After three incorrect attempts, the login button is disabled to prevent further attempts.
- Problem 3: Passing data (e.g., username) between activities.
  - Solution: `Intent` was used with `putExtra` and `getStringExtra` methods to pass the username from `MainActivity` to `LoginSuccessActivity` for displaying a personalized welcome message.

Shortcomings of the App

- Data Security: Currently, the app uses `SharedPreferences` to store the username and password, which is suitable for a basic app but lacks security for a production environment. A more secure storage solution, such as a database, should be considered for real applications.
- Basic User Interface: The current UI is simple and lacks a visually appealing design. It does not provide a great user experience due to its basic layout.

Future Improvements

- Enhance Data Security: Implement a database solution such as SQLite or Firebase to store user credentials, which would increase data security.
- Improve UI/UX: Update the app to include better visual elements, possibly using Material Design or custom layouts, to create a more engaging user interface.

- **Additional Features**: Add a "Forgot Password" feature to allow users to reset their password upon identity verification. Additionally, implement password encryption to further enhance security.